PEARLS AQI PREDICTOR

AI-Powered Air Quality Forecasting System for Lahore, Pakistan

Project Documentation & Technical Report

Prepared by

Talha Shahzad

10Pearls Shine Internship Program — Data Science – Cohort 9

GitHub Repository: github.com/MtalhaShzd/10Pearls_AQI_Predictor

July 2026 – September 2026 · 8 Weeks · Internship Program

Table of Contents

1. Executive Summary

The Pearls AQI Predictor is an end-to-end, production-oriented machine learning system that forecasts the Air Quality Index (AQI) for Lahore, Pakistan across 24-hour, 48-hour, and 72-hour horizons. The project was developed as part of the 10Pearls Shine Internship Program and implements the complete lifecycle of a modern MLOps system: automated hourly data ingestion, feature engineering, a managed feature store, comparative model training, model registry management, SHAP-based explainability, a REST API, and an interactive dashboard — all orchestrated through scheduled CI/CD automation.

Beyond the core forecasting pipeline, this project involved substantial real-world systems engineering: diagnosing and resolving a live third-party compute quota exhaustion incident, hardening the system against unreliable external APIs and unreliable CI scheduling, and building a multi-layer fallback architecture so the application degrades gracefully rather than failing outright when upstream services are unavailable. This report documents the full technical implementation, the design rationale behind each component, and — in detail — the operational challenges encountered during development and how each was diagnosed and resolved.

The final production model (Ridge Regression, selected after comparison against a persistence baseline, Random Forest, XGBoost, and an LSTM) was cross-validated against Punjab EPA's official government air-quality monitoring network and found to be within approximately 4% of the official reading on the day of validation, while the system was simultaneously operating in a degraded fallback mode due to an ongoing third-party service outage — demonstrating both predictive accuracy and operational resilience under real conditions.

2. Project Overview & Objectives

2.1 Problem Statement

Air quality in Lahore fluctuates significantly on an hourly basis due to weather conditions, traffic patterns, industrial activity, and seasonal smog. Residents and decision-makers benefit from forward-looking AQI forecasts, not just current readings, in order to plan outdoor activity and take precautionary measures during hazardous periods.

2.2 Objectives

- Build an automated pipeline that continuously ingests weather and pollutant data with no manual intervention.
- Engineer temporal, cyclical, lag, and rolling features suited to short-horizon AQI forecasting.
- Store and version features using a managed feature store (Hopsworks).
- Train and objectively compare multiple modelling approaches, selecting the best on validation performance.
- Provide model explainability (SHAP) so predictions are interpretable, not just accurate.
- Generate recursive 72-hour forecasts and serve them through a REST API and an interactive dashboard.
- Automate the entire pipeline (hourly ingestion, daily retraining) through CI/CD.
- Ensure the system remains functional and transparent to the user even when third-party infrastructure (feature store, weather API) is degraded or unavailable.

2.3 Scope

The system targets a single city (Lahore) as a proof of concept, using free-tier infrastructure throughout (Hopsworks Serverless, Render, Streamlit Community Cloud, GitHub Actions, Open-Meteo). This constraint — deliberately working within free-tier resource limits — became a significant and instructive part of the project, discussed at length in Section 8.

3. System Architecture

The system follows a layered MLOps architecture separating data ingestion, feature storage, model training, and serving into independently scheduled and independently fault-tolerant stages.

3.1 High-Level Data Flow

- External APIs (Open-Meteo Weather + Air Quality) → Hourly Feature Pipeline (GitHub Actions)
- Feature Pipeline → Feature Engineering → Hopsworks Feature Store (streaming, online + offline) and local CSV (git-committed, CI-persisted)
- Daily Training Pipeline → reads freshest available feature data → trains & evaluates candidate models → registers best model in Hopsworks Model Registry
- Forecast Engine → loads model + features → recursive 72-step prediction → SHAP explanation
- Flask REST API → exposes current conditions, forecasts, and SHAP explanations
- Streamlit Dashboard → consumes the REST API → renders interactive visualizations

3.2 Deployment Topology

Component	Platform	Notes
Flask REST API	Render (Web Service)	Runs via Gunicorn; auto-deploys on push to main
Streamlit Dashboard	Streamlit Community Cloud	Auto-deploys on push to main
Feature Store & Model Registry	Hopsworks Serverless	Managed platform (not self-hosted)
CI/CD Automation	GitHub Actions	Scheduled and manually-triggerable workflows

3.3 Live Endpoints

API (Render): https://one0pearls-aqi-predictor.onrender.com

Dashboard (Streamlit Cloud): https://10pearls-lahore-aqi-predictor.streamlit.app

Source Repository: https://github.com/MtalhaShzd/10Pearls_AQI_Predictor

4. Methodology — Pipeline Stages

Development proceeded through nine major stages, each documented in a corresponding Jupyter notebook in the repository's notebooks/ directory, and subsequently productionized into the automated scripts described in Section 5.

Stage 1 — Data Collection

Notebook: 01_data_collection.ipynb

Historical weather and pollutant data was collected from January 2024 onward using the Open-Meteo Weather API and Open-Meteo Air Quality API for Lahore's coordinates (31.5204° N, 74.3587° E). Weather variables collected: temperature, relative humidity, surface pressure, precipitation, cloud cover, wind speed, and wind direction. Pollutant variables collected: PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone, and the US AQI index.

Stage 2 — Data Preprocessing

Notebook: 02_data_preprocessing.ipynb

Raw data was validated and cleaned through timestamp validation, duplicate removal, missing-value checks, hourly interval continuity validation, data type enforcement, and chronological ordering. This stage established the baseline dataset integrity checks that the production pipelines (Section 5) later automated and re-applied on every hourly run.

Stage 3 — Exploratory Data Analysis

Notebook: 03_EDA.ipynb

Exploratory analysis examined AQI distribution, pollutant inter-correlations, weather–AQI relationships, and hourly, daily, and seasonal AQI patterns. This analysis directly informed the feature engineering choices in Stage 4 — in particular, the strong short-term autocorrelation observed in AQI motivated the inclusion of lag and rolling features.

Stage 4 — Feature Engineering

Notebook: 04_feature_engineering.ipynb

Four categories of engineered features were constructed:

- Temporal: hour, day, month, day_of_week, is_weekend
- Cyclical (sine/cosine encodings to represent periodicity without artificial discontinuity): hour_sin, hour_cos, month_sin, month_cos
- Lag features: aqi_lag_1h, aqi_lag_24h

- Rolling statistics: aqi_rolling_mean_6h, aqi_rolling_std_6h, pm25_rolling_mean_6h
- Change/momentum feature: aqi_change_rate

These 29 total features allow the model to capture both short-term AQI momentum and longer daily/seasonal structure.

Stage 5 — Hopsworks Feature Store

Notebook: 05_hopsworks_feature_store.ipynb

Engineered features are persisted to a Hopsworks Feature Store feature group (lahore_air_quality_features, version 2) configured with datetime as both the primary key and event-time column, with both online (RonDB, low-latency key-value) and offline (Delta/Parquet, queryable via Spark) storage enabled. The feature group is configured for streaming ingestion (stream=True), meaning writes are delivered via Kafka rather than a direct batch write — a design decision that later proved central to diagnosing a production incident (Section 8.3).

Stage 6 — Model Training & Comparison

Notebook: 06_model_training.ipynb

Five approaches were evaluated using a chronological (non-shuffled) 70/15/15 train/validation/test split, chosen deliberately over random shuffling to reflect realistic forecasting conditions where the model only ever has access to the past:

- Persistence baseline (naive: next hour = current hour)
- Ridge Regression (with StandardScaler)
- Random Forest
- XGBoost
- LSTM (deep sequence model)

Stage 7 — Model Registry

Notebook: 07_model_registry.ipynb

The selected production model is registered in the Hopsworks Model Registry, which manages model versions, artifacts, metrics, and metadata. The registration step is gated by an --only-if-better flag in the daily retraining pipeline (Section 5.2), which compares the newly trained model's test RMSE against the best previously registered version and only registers a new version when it represents a genuine improvement — deliberately preventing unbounded version sprawl from daily retraining.

Stage 8 — SHAP Explainability

Notebook: 08_shap_explainability.ipynb

SHAP (SHapley Additive exPlanations) is used to attribute each forecast to its contributing input features, answering "why did the model predict this AQI?" for any selected forecast horizon. Implementing this correctly in production required resolving a non-trivial technical issue described in Section 8.1.

Stage 9 — Recursive 72-Hour Forecasting

Notebook: 09_forecasting_pipeline.ipynb

The forecast engine generates predictions recursively: the model predicts AQI at t+1, that prediction is fed back into the lag/rolling features to construct the input for t+2, and so on through t+72. Future weather inputs (which the model also depends on) are sourced from the Open-Meteo Forecast API. This recursive dependency structure — where later predictions partially depend on the model's own earlier predictions — is an important characteristic discussed further in Section 6.3.

5. Production Pipelines & Automation

5.1 Hourly Feature Pipeline

Script: scripts/feature_pipeline.py — Workflow: .github/workflows/hourly.yml

Runs on an hourly GitHub Actions schedule. Fetches the most recent weather and pollutant data, engineers features, appends to the local processed CSV, and upserts new/refreshed rows into the Hopsworks feature group via Kafka-based streaming ingestion. A 24-hour overlap window is re-sent on each run so that lag/rolling features self-correct as new ground-truth data arrives.

5.2 Daily Training Pipeline

Script: scripts/training_pipeline.py — Workflow: .github/workflows/daily.yml

Runs once daily (02:00 UTC / 07:00 PKT). Loads the freshest available feature data, retrains a scaled Ridge Regression model, evaluates it against a persistence baseline, and registers it in the Model Registry only if it improves on the best existing version.

5.3 Backfill Pipeline

Script: scripts/backfill_pipeline.py — Workflow: .github/workflows/backfill.yml

Automatically detects missing hourly timestamps in the dataset, fetches the missing data, recomputes lag/rolling features across the affected range, and upserts the correction. Originally a manually-triggered utility, it was later converted into a scheduled safety net (Section 8.4) that runs automatically to catch any hours missed by the hourly pipeline.

5.4 REST API

Framework: Flask — Entry point: api/app.py — Production server: Gunicorn

Endpoint	Method	Purpose
/	GET	Basic health check
/health	GET	Extended service-level health status
/routes	GET	Lists all registered API routes
/api/current	GET	Current AQI, pollutants, weather, 24h trend
/api/forecast	GET	72-hour recursive AQI forecast
/api/shap/<horizon>	GET	SHAP feature contributions for 24h / 48h / 72h
/api/source	GET	Reports which data/model source is currently active (live vs fallback)

Endpoint	Method	Purpose
/api/reload	GET / POST	Forces a fresh read of model and feature data, bypassing cache

5.5 Streamlit Dashboard

Entry point: app/dashboard.py

An interactive dashboard displaying real-time pollutant readings, a 24-hour AQI trend chart, current weather conditions, 24/48/72-hour forecast cards with health classifications, a predicted-AQI trend chart, model performance metrics, a downloadable 72-hour forecast table, and SHAP feature-contribution charts for the selected horizon. The dashboard was substantially hardened for graceful degradation during development, detailed in Section 8.6.

6. Model Performance & Results

6.1 Validation Results

Model	MAE	RMSE	R^2
Ridge Regression (selected)	1.0044	1.5177	0.9988
Persistence Baseline	1.1930	1.8827	0.9981
XGBoost	1.0790	2.4960	0.9966
Random Forest	1.6369	2.6725	0.9961
LSTM	4.3263	5.7691	0.9819

6.2 Final Test Results — Ridge Regression

Metric	Result
MAE	2.8946
RMSE	4.6925
R^2	0.9868
Predictions within ±7.06 AQI	90%
Predictions within ±10.26 AQI	95%

Note: the table above reflects results at the time of initial model selection. Because the daily training pipeline continuously retrains and re-registers the model (subject to the --only-if-better guard), live metrics drift over time and can be retrieved at any point via the /api/forecast or /api/source endpoints. At the time of writing, the production model (v33) reported a live test RMSE of 6.32.

6.3 Why Ridge Regression

Ridge Regression achieved the strongest overall validation performance while remaining computationally lightweight, fast to retrain daily, and — critically — linearly interpretable, which made exact SHAP attribution tractable (Section 8.1). AQI exhibits strong short-term temporal dependence, and the engineered lag and rolling features supplied most of the model's predictive power; a more complex non-linear model was not justified by the marginal accuracy gain, if any. It is worth noting explicitly that because the 72-hour forecast is generated recursively, later-horizon predictions increasingly depend on the model's own earlier predictions via the lag features — meaning forecast uncertainty compounds with horizon length. This is reflected in the horizon-scaled RMSE reported in the dashboard (24h / 48h / 72h) and is an inherent property of recursive forecasting, not a model deficiency.

6.4 External Validation

On August 28, 2026, the system's live current-AQI prediction (166) was cross-checked against Punjab EPA's official government monitoring network, which reported a city-wide AQI of 160 at the same time — a deviation of approximately 4%. This comparison is notable because it was performed while the system was operating in local-CSV fallback mode due to an ongoing Hopsworks compute quota outage (Section 8.3), demonstrating that both the underlying model accuracy and the fallback data pathway independently produced results consistent with an authoritative, independent, ground-truth source.

7. Resilience & Fault-Tolerance Architecture

A defining characteristic of this project is that it was built, tested, and hardened against real production failures encountered during development — not designed defensively in the abstract. This section describes the resulting architecture; Section 8 documents the specific incidents that motivated each layer.

7.1 Feature & Model Freshness Comparison

A critical early finding was that a successful read from the Hopsworks Feature Store does not guarantee fresh data — the offline store can silently fall behind if its materialization job is failing, while reads against it continue to succeed (returning stale data with no error). To address this, get_features_df() (serving the API/dashboard) and load_dataframe() (serving daily training) both read from Hopsworks and the local CI-committed CSV, compare their latest timestamps, and serve whichever is genuinely more current — Hopsworks winning exact ties. Both paths self-heal automatically once Hopsworks catches up, without any manual intervention or restart.

7.2 Model Artifact Fallback with Auto-Retry

Model loading attempts the Hopsworks Model Registry first and falls back to the last locally cached model artifact on failure. Rather than remaining permanently stuck on the fallback, the system retries the Hopsworks Model Registry on a timer (every 30 minutes by default) whenever it is running in a degraded state, switching back automatically the moment the registry becomes reachable again.

7.3 Weather API Resilience (Three-Tier Fallback)

The weather-forecast fetch used by the forecast engine implements three tiers of resilience: (1) a live call to Open-Meteo; (2) a 15-minute in-memory cache, since a single dashboard load or SHAP horizon switch can otherwise trigger several redundant external calls in quick succession; and (3), if both the live call and the cache are unavailable simultaneously — the specific case of a freshly restarted service encountering a rate-limited external API before any cache has been populated — a synthesized flat 72-hour weather input derived from the most recent known local readings, keeping the AQI model runnable rather than failing outright.

7.4 Forecast Computation Caching

The full 72-step recursive forecast computation is cached for 10 minutes, since both the /api/forecast endpoint and every SHAP horizon selection independently trigger the same expensive computation. Caching reduces redundant load on the model and feature store without affecting forecast correctness within the cache window.

7.5 Non-Blocking Pipeline Writes

Hopsworks write failures within the hourly, daily, and backfill pipelines are caught and logged as warnings rather than allowed to crash the pipeline. This ensures that the local CSV — the fallback data source underpinning Section 7.1 — continues to be updated and committed even when Hopsworks itself is degraded or unreachable, preventing the fallback data from also going stale.

7.6 Transparent Degradation

Rather than silently serving fallback data as if it were live, the dashboard surfaces an explicit, user-facing banner whenever any component is operating in a degraded state, along with plain-language messaging that no user action is required and that the system will recover automatically.

8. Challenges, Issues & Resolutions

This section documents, in detail, the significant technical issues encountered during development and productionization, presented as engineering case studies. Several of these reflect genuine production incidents involving third-party infrastructure, not merely local development bugs, and are included to demonstrate the diagnostic process as much as the fix.

8.1 SHAP Explanation Crash on Production Model

Problem: The /api/shap/<horizon> endpoint failed with AttributeError: 'Ridge' object has no attribute 'named_steps' in some environments, while working correctly in others.

Diagnosis: The training pipeline had been updated to wrap the Ridge model in a scikit-learn Pipeline (StandardScaler + Ridge) to correctly scale features of very different magnitudes (e.g. surface_pressure ~1000 vs. hour_sin ~1). shap.LinearExplainer, however, requires a raw linear model exposing coef_/intercept_, not a Pipeline object. Locally cached model artifacts predating this change were bare Ridge models, while the live Hopsworks Model Registry model was already the new Pipeline — producing inconsistent behavior across environments.

Resolution: get_shap_explanation() was updated to detect whether the loaded model is a Pipeline (via hasattr(model, 'named_steps')) and, if so, extract the fitted scaler and Ridge estimator separately, manually transforming the input data through the scaler before passing the raw Ridge estimator to LinearExplainer. A bare-model code path was retained for backward compatibility with older cached artifacts.

Outcome: SHAP explanations now function correctly regardless of whether the loaded model is a raw estimator or a scaler-wrapped Pipeline, in every environment.

8.2 SHAP Explanations Identical Across All Forecast Horizons

Problem: The 24h / 48h / 72h SHAP tabs on the dashboard displayed identical feature-contribution charts regardless of which horizon was selected.

Diagnosis: get_shap_explanation() was explaining only the most recent historical data row, ignoring the horizon parameter entirely — the recursive forecast loop computed a distinct feature vector for every one of its 72 steps, but generate_72h_forecast() discarded those intermediate feature vectors, retaining only the final AQI predictions.

Resolution: generate_72h_forecast() was extended with a return_features flag to optionally return the full per-step feature matrix alongside the predictions. get_shap_explanation(horizon) then indexes into that matrix at the row corresponding to the requested horizon (index 23, 47, or 71) and explains that specific feature vector.

Outcome: Each horizon now produces a genuinely distinct SHAP explanation, correctly reflecting that the 72-hour forecast increasingly relies on the model's own earlier predictions (via lag features) as the horizon lengthens.

8.3 Hopsworks Compute Quota Exhaustion (Multi-Day Production Incident)

Problem: The dashboard's "Last Data Ingested" timestamp stopped advancing and remained frozen for several days, despite the hourly GitHub Actions pipeline reporting successful runs throughout.

Diagnosis: Investigation proceeded systematically: (1) confirmed the Render API service and Streamlit dashboard were both functioning correctly; (2) confirmed /api/current successfully read from Hopsworks and returned a plausible row count; (3) cross-referenced against GitHub Actions logs, which showed the hourly pipeline successfully sending new rows via Kafka on every run; (4) traced the discrepancy to the Hopsworks Jobs UI, which revealed the lahore_air_quality_features_2_offline_fg_materialization job — the step that commits streamed data into the queryable offline store — failing on every execution for several consecutive days, each failure completing in approximately 8 seconds versus the 2-3 minutes a healthy run required. This instant-failure signature indicated rejection at job submission rather than a data or processing error, which was confirmed by a billing notification indicating the Hopsworks Serverless monthly compute credit had been exhausted.

Resolution: Multiple mitigations were implemented: get_features_df() and load_dataframe() were rewritten to compare timestamps between Hopsworks and the local CSV and serve whichever was genuinely fresher (Section 7.1); the hourly/daily/backfill pipelines were made resilient to write failures so the local CSV fallback itself never went stale (Section 7.5); and the dashboard was given an explicit degraded-mode indicator so the fallback state was visible rather than silent.

Outcome: The application remained fully functional throughout the multi-day outage, correctly serving current (same-day) data from the local CSV fallback while Hopsworks' offline store remained frozen — validated directly against Punjab EPA's official AQI reading during this exact window (Section 6.4).

8.4 GitHub Actions Scheduled Workflows Silently Missing Runs

Problem: The hourly feature pipeline was observed to have an 11-hour gap between consecutive scheduled runs, despite the workflow configuration specifying an hourly cron schedule with no errors reported.

Diagnosis: This is a documented GitHub Actions platform limitation: scheduled (cron) workflow triggers are explicitly best-effort and can be delayed or, during periods of high platform load, dropped entirely — with round-number trigger minutes (:00, :15, :30, :45) experiencing the heaviest contention, since the majority of scheduled workflows across GitHub default to those times.

Resolution: The hourly workflow's cron minute was moved off a round number (from :15 to :23) to reduce contention. More importantly, the previously manual-only backfill pipeline — which already auto-detects and fills any gap in the hourly time series — was converted into a scheduled workflow running independently, acting as a safety net that requires no round-number alignment with the primary hourly job.

Outcome: Any run missed by the primary hourly pipeline is now automatically detected and backfilled by the independent scheduled backfill run within its own interval, without requiring manual intervention or even visibility into whether the primary pipeline succeeded.

8.5 Open-Meteo Rate Limiting (HTTP 429) on the Forecast Endpoint

Problem: /api/forecast intermittently returned HTTP 500 errors, with the underlying cause obscured because Flask's exception handling converted the real error into a generic 500 response.

Diagnosis: Fetching the raw JSON response body directly (bypassing the dashboard's generic error display) revealed the actual error: HTTP Error 429: Too Many Requests from Open-Meteo. The forecast engine called the weather API on every single request to /api/forecast and every SHAP horizon selection

with no caching, and Render's shared outbound IP pool compounds this risk across unrelated applications sharing the platform.

Resolution: fetch_weather_forecast() was rewritten with a 15-minute in-memory cache, and — since a cache is empty immediately after every service restart, which could still coincide with an active rate-limit window — a further fallback synthesizes a flat weather forecast from the most recent locally known weather readings when both the live API and the cache are unavailable.

Outcome: The forecast endpoint no longer fails outright due to upstream rate limiting under normal operation, and remains functional (with clearly degraded weather inputs, while AQI-driving lag features remain live) even in the narrow cold-start edge case.

8.6 Dashboard Cascading Failures on Partial API Outages

Problem: Early versions of the dashboard treated the current-conditions and forecast API calls as a single all-or-nothing unit: if the forecast endpoint failed for any reason, the entire dashboard displayed only a generic error and rendered nothing else, even though current conditions were available and functioning correctly.

Diagnosis: The dashboard's control flow used a single combined check (if not current or not forecast) that halted rendering (st.stop()) before any content was drawn, rather than degrading section-by-section.

Resolution: The dashboard was restructured so that a missing current-conditions response remains a hard stop (since nearly every other section depends on it), while a missing forecast response now shows a specific, actionable warning and allows the pollutant cards, 24-hour trend, and current-conditions sections to render normally; a SHAP-specific failure similarly degrades only the SHAP section rather than the whole page.

Outcome: The dashboard now presents the maximum available information under any partial-failure condition, rather than an all-or-nothing outage.

8.7 CI/CD Workflow Push Logic Silently Broken

Problem: A manual review of the daily training workflow revealed that its git-push retry loop was missing its actual push command — the loop printed retry messages and always failed, meaning the workflow would have failed to commit updated model metadata on every run without any change in behaviour actually being attempted.

Diagnosis: Comparison against the equivalent, correctly functioning logic in the hourly and backfill workflows confirmed the missing line (git pull --rebase --autostash origin main && git push origin main && exit 0) had been dropped, likely during a prior manual edit.

Resolution: The missing command was restored, and all three CI/CD workflow files (hourly, daily, backfill) were subsequently cross-checked line-by-line against one another to confirm consistency.

Outcome: This case underscores the value of systematically re-verifying automation scripts after manual edits, rather than assuming a workflow that runs without an explicit error is behaving as intended.

8.8 Feature Group Version Inconsistency

Problem: The project migrated its Hopsworks feature group from a non-streaming version (v1) to a streaming-enabled version (v2) partway through development, but the default fallback value for the feature group version environment variable was inconsistent across the codebase — most files correctly defaulted to "2", but src/hopsworks_store.py still defaulted to "1".

Diagnosis: A repository-wide search confirmed the inconsistent default, along with two standalone scripts (a debug utility and a superseded backfill implementation) still hardcoding version=1 directly rather than reading the environment variable at all.

Resolution: All defaults were unified to version 2, and the hardcoded references were corrected. The obsolete v1 feature group was subsequently deprecated and then deleted from Hopsworks once it was confirmed nothing in the active codebase referenced it.

Outcome: This was caught proactively through a defensive repository audit before deleting the old feature group, rather than being discovered reactively through a production failure — illustrating the value of verifying via search rather than assuming consistency across a codebase that evolved incrementally.

9. Deployment

9.1 Render (REST API)

The Flask API is deployed as a Render Web Service, served in production via Gunicorn (gunicorn api.app:app --timeout 300 --workers 1 --threads 4 --graceful-timeout 120) rather than Flask's built-in development server. Render injects the service's listening port via the PORT environment variable at runtime, which the application binds to dynamically. The service auto-deploys on every push to the main branch.

9.2 Streamlit Community Cloud (Dashboard)

The Streamlit dashboard is deployed on Streamlit Community Cloud, also auto-deploying on push to main, and reads the backend API's base URL from an environment variable at runtime, defaulting to the Render deployment above if unset.

9.3 Hopsworks Serverless (Feature Store & Model Registry)

Feature storage and model registry are hosted on Hopsworks' managed Serverless platform rather than self-hosted infrastructure. As documented in Section 8.3, this platform's shared, credit-metered compute model was directly responsible for a significant production incident during development.

9.4 GitHub Actions (CI/CD)

All scheduled automation (hourly ingestion, daily retraining, periodic backfill) runs on GitHub Actions against the public repository, which provides unlimited included minutes for public repositories.

9.5 Free-Tier Operating Constraints

Every platform in this stack was deliberately used at its free tier. Render and Streamlit Cloud both spin down idle services after a period of inactivity, producing a cold-start delay (typically 30-50 seconds) on the first request after idle time; the dashboard surfaces this explicitly to the user rather than presenting it as an error. Hopsworks Serverless's compute credit model was the direct cause of the incident in Section 8.3. These constraints are treated throughout this report not as external excuses but as legitimate engineering conditions the system was explicitly designed and tested to tolerate.

10. Testing & Verification

Verification was performed through a combination of local scripted checks and live production monitoring, given the system's dependence on external, sometimes-unreliable third-party services.

- Local SHAP verification: a standalone script (test_shap.py) confirmed that all three forecast horizons produce genuinely distinct SHAP feature-contribution rankings after the fix described in Section 8.2.
- Endpoint-level verification: each API endpoint (/api/current, /api/forecast, /api/shap/<horizon>, /api/source, /api/reload) was manually exercised via curl against the live Render deployment to confirm correct behaviour, including under the degraded conditions described in Section 8.
- CI/CD verification: each GitHub Actions workflow was manually triggered via workflow_dispatch and its logs reviewed end-to-end at least once following every significant change, rather than relying solely on the next scheduled run.
- External validation: the live current-AQI prediction was cross-checked against Punjab EPA's official government monitoring dashboard (Section 6.4).
- Resilience verification: the freshness-comparison fallback, weather-cache fallback, and degraded-mode dashboard banner were each confirmed operating correctly under a genuine, live Hopsworks outage (Section 8.3), rather than only under simulated conditions.

11. Future Improvements

- Automated model drift detection and data quality monitoring, alerting when live model performance diverges meaningfully from validation-time metrics.
- Automated rollback to the previous registered model version if a newly registered model underperforms in production.
- Direct read access to Hopsworks' online (RonDB) feature store for latency-sensitive queries, which may sidestep the offline-store materialization dependency responsible for the incident in Section 8.3.
- Extension to additional cities and geographic regions beyond Lahore.
- Incorporation of weather forecast uncertainty (rather than point estimates) into the recursive forecasting loop.
- A formal automated test suite (the tests/ directory currently exists as a placeholder) covering the pipelines and API independent of manual/live verification.

12. Conclusion

The Pearls AQI Predictor successfully demonstrates a complete, automated, and explainable machine learning system for short-horizon air quality forecasting, covering the full MLOps lifecycle from raw data ingestion through model serving and explainability. Model selection was performed rigorously through comparative evaluation, and the chosen Ridge Regression model was independently validated against an authoritative government data source with strong agreement.

Equally significant to the modelling work is the operational resilience engineered into the system over the course of development, in direct response to real production incidents encountered on free-tier third-party infrastructure — a compute quota exhaustion on the feature store, unreliable external scheduling, and rate-limiting on an external weather API. Rather than treating these as one-off bugs to patch silently, each was diagnosed methodically, documented, and addressed with a durable, self-healing mechanism, resulting in a system that degrades gracefully and recovers automatically without manual intervention. This combination of predictive modelling and production-grade fault tolerance reflects the practical, end-to-end MLOps skill set the 10Pearls Shine Internship Program was designed to develop.

Appendix

A. Repository & Resources

Source Repository: https://github.com/MtalhaShzd/10Pearls_AQI_Predictor

Notebooks Directory: https://github.com/MtalhaShzd/10Pearls_AQI_Predictor/tree/main/notebooks

Live API: https://one0pearls-aqi-predictor.onrender.com

Live Dashboard: https://10pearls-lahore-aqi-predictor.streamlit.app

B. Notebook Index

Notebook	Purpose
01_data_collection.ipynb	Initial weather and AQI data collection
02_data_preprocessing.ipynb	Cleaning and validation
03_EDA.ipynb	Exploratory data analysis
04_feature_engineering.ipynb	Feature creation
05_hopsworks_feature_store.ipynb	Feature Store integration
06_model_training.ipynb	Model comparison and evaluation
07_model_registry.ipynb	Model registration
08_shap_explainability.ipynb	SHAP explainability
09_forecasting_pipeline.ipynb	72-hour forecasting

C. Technology Stack

- Languages & Data: Python, SQL-style data-processing workflows
- Machine Learning: Scikit-learn, XGBoost, TensorFlow/Keras, SHAP
- MLOps: Hopsworks Feature Store, Hopsworks Model Registry, GitHub Actions
- Backend: Flask, Gunicorn, REST API
- Frontend: Streamlit, Plotly
- Data Sources: Open-Meteo Weather API, Open-Meteo Air Quality API
- Deployment: Render, Streamlit Community Cloud
- Tooling: VS Code, Jupyter Notebook, Git, GitHub